

Nomadic Online Traveling Salesman on the Ring

Jatinder Sahota, Dr. Stephane Durocher

Department of Computer Science, University of Manitoba

Background

The Traveling Salesman Problem (TSP) is a classical optimization problem in which a salesman (server) must visit (serve) a set of points (requests) exactly once, with the objective is minimizing total travel distance. In the *online* variant, requests appear over time, and the salesman must make decisions without knowledge of future requests. The problem was coined by Ausiello et al. (1999).

We consider the **nomadic** online TSP on the **ring** (N-OLTSP-R), where points lie on a circle of circumference C and distances are measured along the arc. In this setting, the server moves at unit speed and is not required to return to their starting point after a tour. Applications include scheduling, robotics and computer networks.

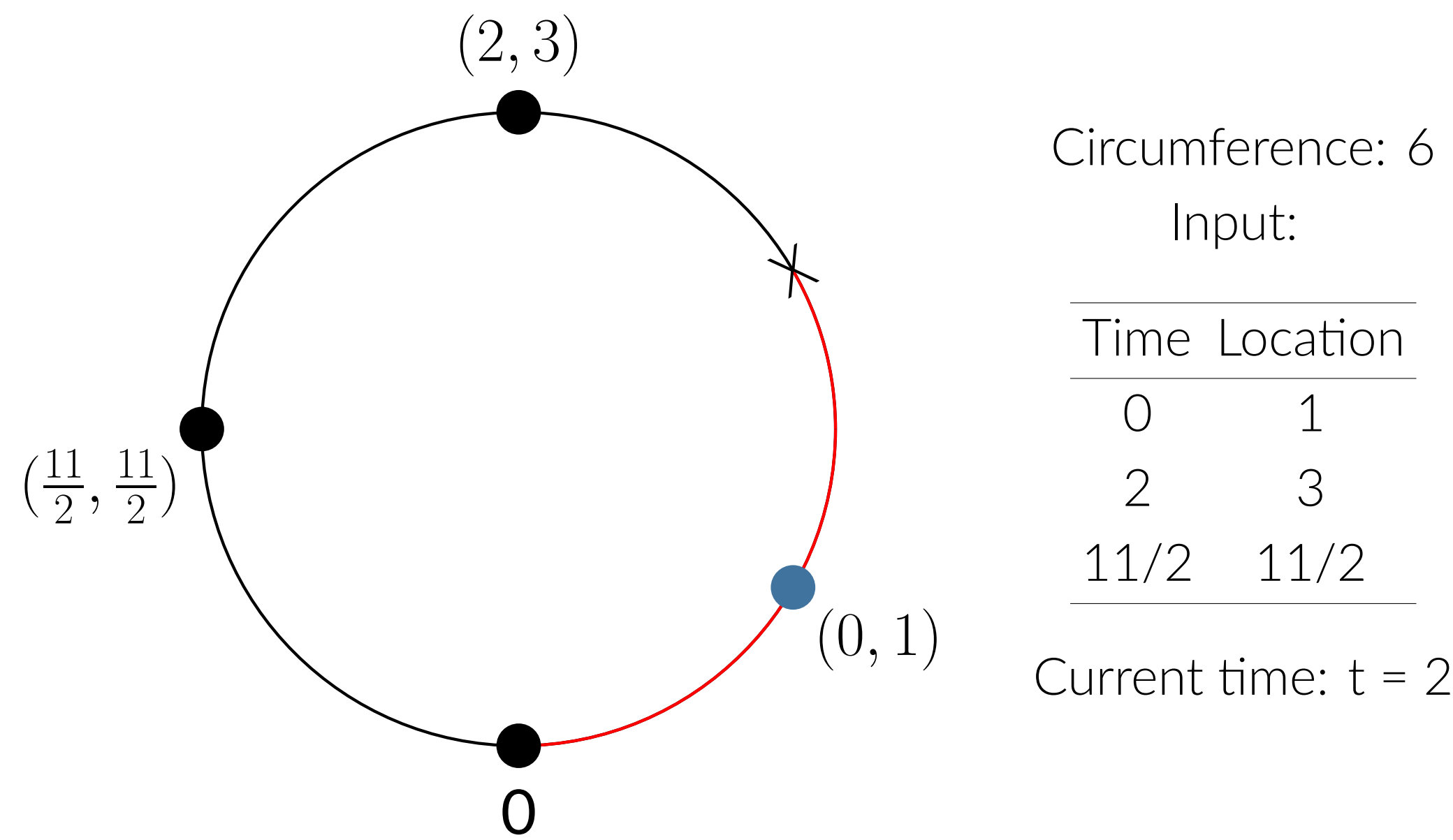


Figure 1. An example; requests are labeled as (time, location) pairs, and the server is the cross.

Definition: A request sequence (σ) of size $M \in \mathbb{N}$ is defined to be:

$$\sigma := \{\sigma_i = (t_i, a_i) : 0 \leq a_i < C, t_i \geq 0\}_{i=1}^M$$

where C is the circumference of the ring. $\square$

To serve a request σ_i , the server must visit a_i at time $t \geq t_i$. We define the "cost" of our algorithm to be the total time spent to serve M -many requests, rather than total travel distance.

Definition: For all request sequences σ , the **competitive ratio** (ρ) of an online algorithm ALG is defined as:

$$\rho := \sup_{\sigma} \frac{Z^{ALG}}{Z^{OPT}}$$

where OPT is the optimal algorithm, and Z^{ALG} and Z^{OPT} are the costs of ALG and OPT on input σ , respectively. $\square$

Definition: Let $d(x, y) :=$ the length of the shorter arc connecting points x and y . $\square$

Previous Work

The following works showcase the state of the problem:

- **Jawgal et al. (2019)** prove a lower bound of $\rho \geq 2$ on the competitive ratio.
- **Jawgal et al. (2019)** find an algorithm with competitive ratio $\rho = 2.5$.
- **Blom et al. (2000)** show that in general, waiting is better than being zealous (not waiting).

An Algorithm for N-OLTSP-R

The current best known algorithm does not wait by construction. A natural direction for improvement is to let the server idle while there are requests to serve.

To ensure the algorithm makes the best decision, the server waits as long as possible before serving the most recent released request, and follows a locally-optimal tour.

Smartly Wait And Go is called when a request outside the current tour is released.

Input: All currently-unserved requests $\sigma(t)$, the current time t , the current position of the online server $p(t)$.

Output: The tour for the server to walk (T^{OL}).

- 1: $I(t) :=$ shortest interval covering all unserved requests at time t .
- 2: $T^G(t) :=$ tour that maximally waits from time 0 and optimally serves $I(t)$ from origin.
- 3: $Z^G(t) :=$ cost of $T^G(t)$.
- 4: $t_0 := 2Z^G(t) - d(p(t), e(t)) - \ell(t) \quad \triangleright$ Latest time to follow T^G so that $\rho = 2$
- 5: **If** $t_0 \geq t$ (i.e. time t_0 has not passed) **then:**
- 6: $T^{OL} :=$ wait until t_0 then walk $I(t)$ through $e(t)$.
- 7: **Else**
- 8: $T^{OL} :=$ minimum-cost tour of: {serve $I(t)$ through $\sigma^L(t)$, serve $I(t)$ through $\sigma^R(t)$ }
- 9: **return** T^{OL}

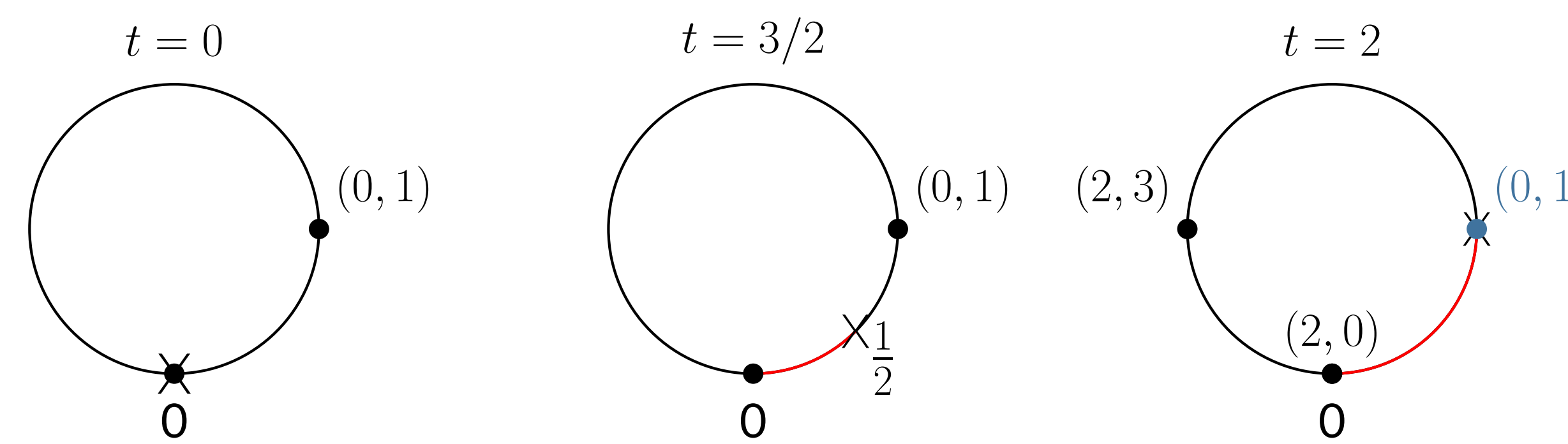


Figure 2. An example demonstrating the algorithm. The server begins moving at $t = 1$.

Definition: Let $I(t) = [\sigma^L(t), \sigma^R(t)]$ be the smallest interval (arc) covering all unserved requests at time t with endpoints $\sigma^L(t) = (t_L, a_L)$ and $\sigma^R(t) = (t_R, a_R)$. Denote the length of the arc as $\ell(t)$, and the most recent-released request $\sigma_m = (t_m, a_m)$. $\square$

Definition: Let $\mathcal{T}(t)$ be the set of tours starting at the origin that serve $I(t)$. Let $w(T)$ denote the amount of time any tour T idles at the origin starting from time 0. Define $T^G(t)$ as:

$$T^G(t) := \arg \max_{T \in \mathcal{T}(t)} \{w(T) : T \text{ serves } I(t) \text{ optimally}\}$$

The first endpoint of $I(t)$ that $T^G(t)$ visits will be denoted $e(t)$. $\square$

Definition: Let t_0 denote the time in execution such that:

$$t_0 + d(p(t), e(t)) + \ell(t) = 2 \times Z^G(t)$$

where $Z^G(t)$ is the cost of $T^G(t)$ and $p(t)$ is the position of the server at time t . $\square$

Competitive Analysis

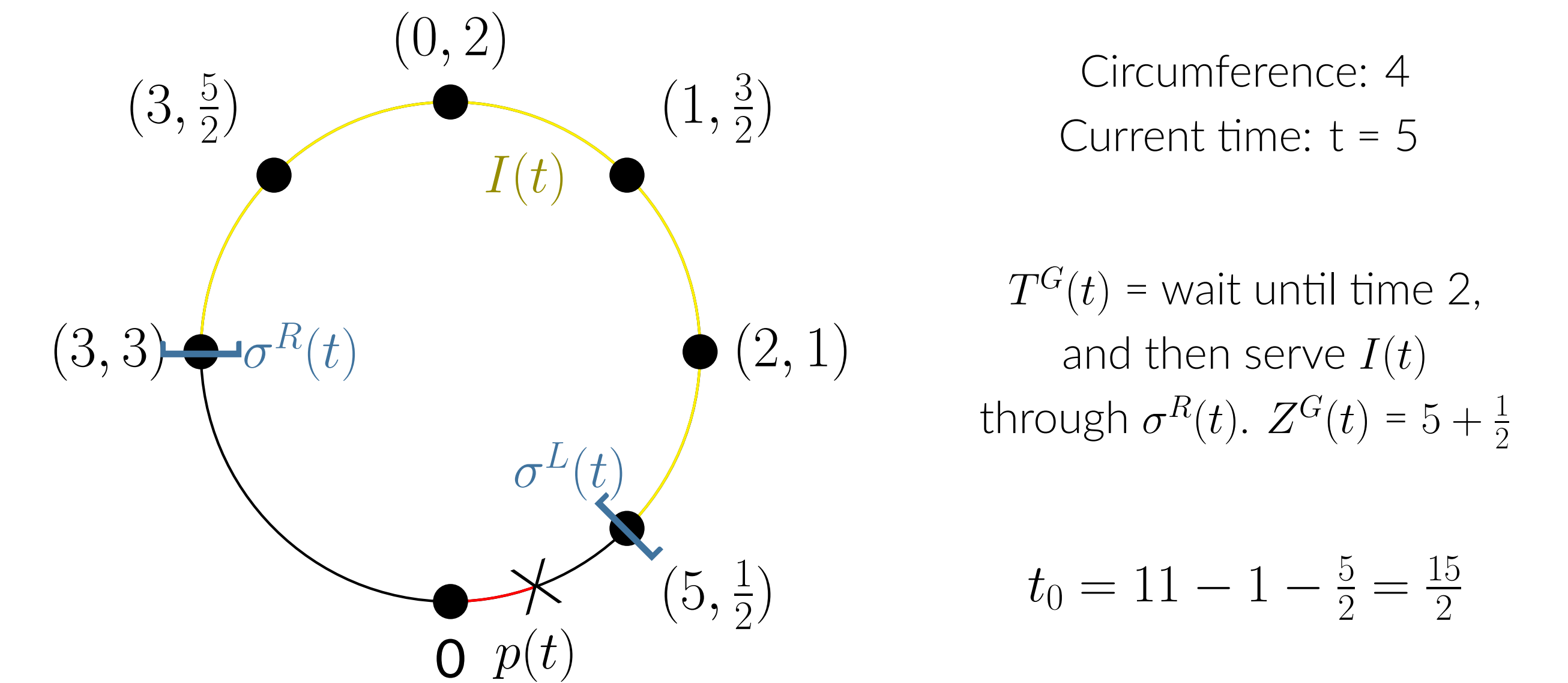


Figure 3. An example illustrating the definitions given before. The yellow arc represents the interval $I(t)$, and the endpoints are labeled. $p(t)$ denotes the position of the server at time $t = 5$.

Lemma 1: For all t and any request sequence, $Z^G(t) \leq Z^{OPT}(t)$, where $T^{OPT}(t)$ optimally serves all requests released up to time t .

Proof sketch: Follows from the definition of $T^G(t)$ and the fact that $T^{OPT}(t)$ must serve at least all of $I(t)$. $\square$

Lemma 2: Let $c(t) = \min\{d(p(t), \sigma^R(t)), d(p(t), \sigma^L(t))\}$. If $t > t_0$ and $L(t) := (t - t_0) + c(t) - d(0, e(t)) > 0$, then,

$$Z^{ALG}(t) = 2 \times Z^G(t) + L(t)$$

Proof sketch:

- $t - t_0$ is how much time has passed since time t_0 .
- $c(t) - d(0, e(t))$ is the difference in distance that our server travels to the closer endpoint compared to $T^G(t)$ and $e(t)$. $\square$

Conjecture: Smartly Wait And Go is 2-competitive.

Intuition: If $t \leq t_0$, then we can afford to idle. Hence, $p(t_0) = p(t)$ and $I(t_0) = I(t)$. Applying the definition of t_0 and Lemma 1:

$$Z^{ALG}(t) = t_0 + d(p(t), e(t)) + \ell(t) = 2 \times Z^G(t) \leq 2 \times Z^{OPT}(t)$$

Otherwise, $t > t_0$, we need only consider the instance constructed in Lemma 2, since otherwise we avoid being late by serving $I(t)$ through the nearer endpoint.

Intuitively, for the server to be placed in such a situation, requests must have been placed prior that moved the server to their current position. We can offset these extra costs since OPT will serve those requests too.

Conclusion and Future Directions

This problem encapsulates many real-world scenarios, and so different constraints can allow for optimizing certain tasks. While the nomadic instance on the ring is close to being solved, some other interesting directions would be:

- Exploring constraints concerning a clairvoyant server.
- Investigating application through learning-augmented algorithms.
- Tightening bounds for various metric spaces.